SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

C0 4 ASSESSMENT TOOL 1

Course Code:	ITA0609	**Course Title:**	Machine Learning
CO Assessed:	CO 4 – Analyze (BL 3) 40% of CIA	**Assessment:**	Assessment Tool 1 30 (3 Questions ×10 Mark)
Weightage:		**Total Marks:**	
CO1 Statement:	*To acquire a foundational understanding of key machine learning concepts including version spaces, candidate elimination, inductive bias, and heuristic search (BL1)*		
Student Name:	**Sameer Ahmed G**	**Reg. No.:**	**192421154**
Section / Batch:	SLOT A	**Date:**	12-08-2026

Code of Conduct

I (Sameer Ahmed G-192421154) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

1.Write a Python program to implement the K-Nearest Neighbour (KNN) algorithm for a small dataset. Use Euclidean distance to find nearest neighbors and classify a test instance. Allow the user to input the value of K and display predicted class labels with accuracy.

K-Nearest Neighbour (KNN)

Concept

- KNN is a **lazy learning algorithm** that classifies a new data point based on the majority class of its nearest neighbors.
- Distance metric: **Euclidean distance** is commonly used to measure closeness.
- The parameter **K** decides how many neighbors are considered.
  - Small K → sensitive to noise.
  - Large K → smoother decision boundary.

Steps

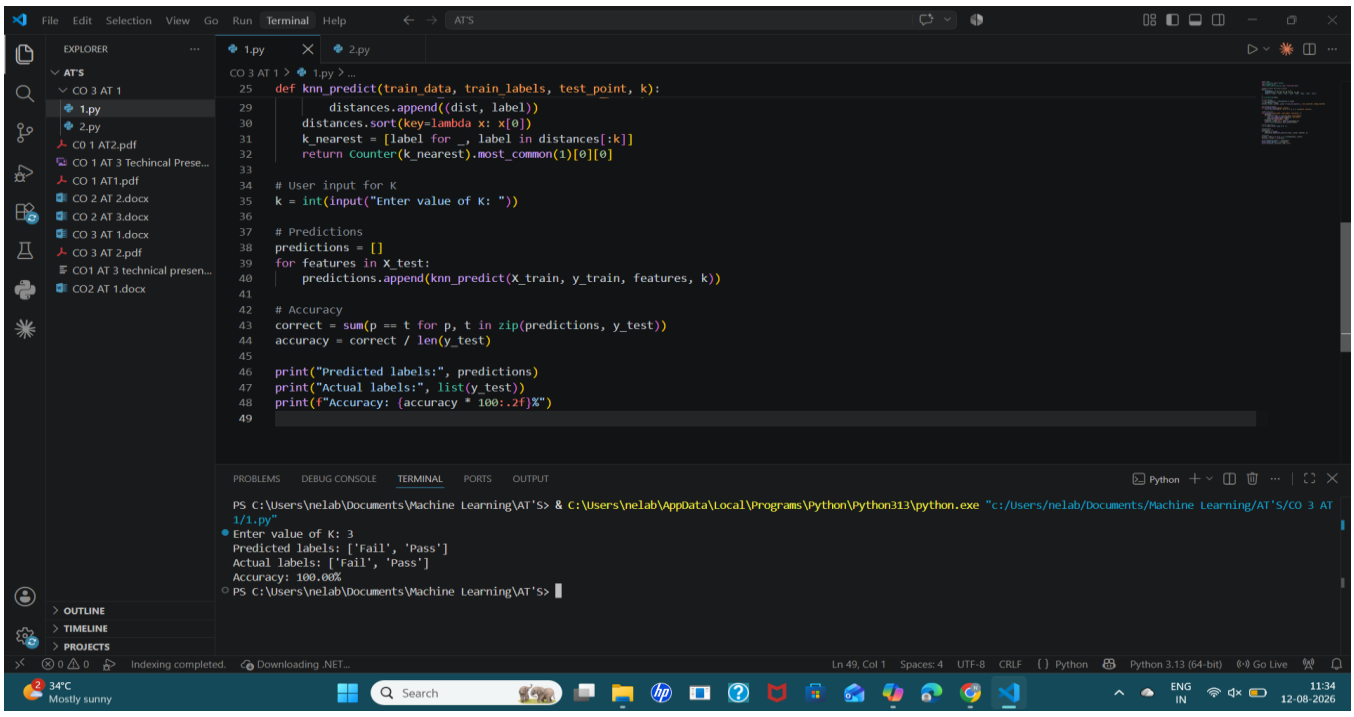
1. Store the dataset (StudyHours, PreviousScore, Result).
2. For a test student, compute Euclidean distance to all training students.
3. Sort distances and pick the **K nearest neighbors**.
4. Use majority voting to assign the class (Pass/Fail).
5. Evaluate predictions against actual labels to compute **accuracy**.

Advantages

- Simple and intuitive.
- Works well for small datasets.
- No training phase, only prediction.

Limitations

- Computationally expensive for large datasets.
- Sensitive to irrelevant features and scaling.



2.Write a Python program to implement **Logistic Regression** for binary classification. Train the model using gradient descent and predict outputs for test data. Display the sigmoid function output and classification results. Evaluate the model using accuracy and discuss how learning rate affects convergence and performance.

Logistic Regression

Concept

- Logistic Regression is a **linear model for binary classification**.
- It uses the **sigmoid function** to map input values into probabilities between 0 and 1.
- Decision rule:
  - Probability ≥ 0.5 → Class 1 (Pass)
  - Probability < 0.5 → Class 0 (Fail)

Steps

1. Initialize weights and bias.
2. Compute linear combination: $z = w \cdot X + b$.
3. Apply sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$.
4. Update weights using **gradient descent**:
   - $w = w - \alpha \cdot dw$
   - $b = b - \alpha \cdot db$where $\alpha$is the learning rate.
5. Repeat for multiple epochs until convergence.
6. Predict labels and evaluate accuracy.

Learning Rate Effect

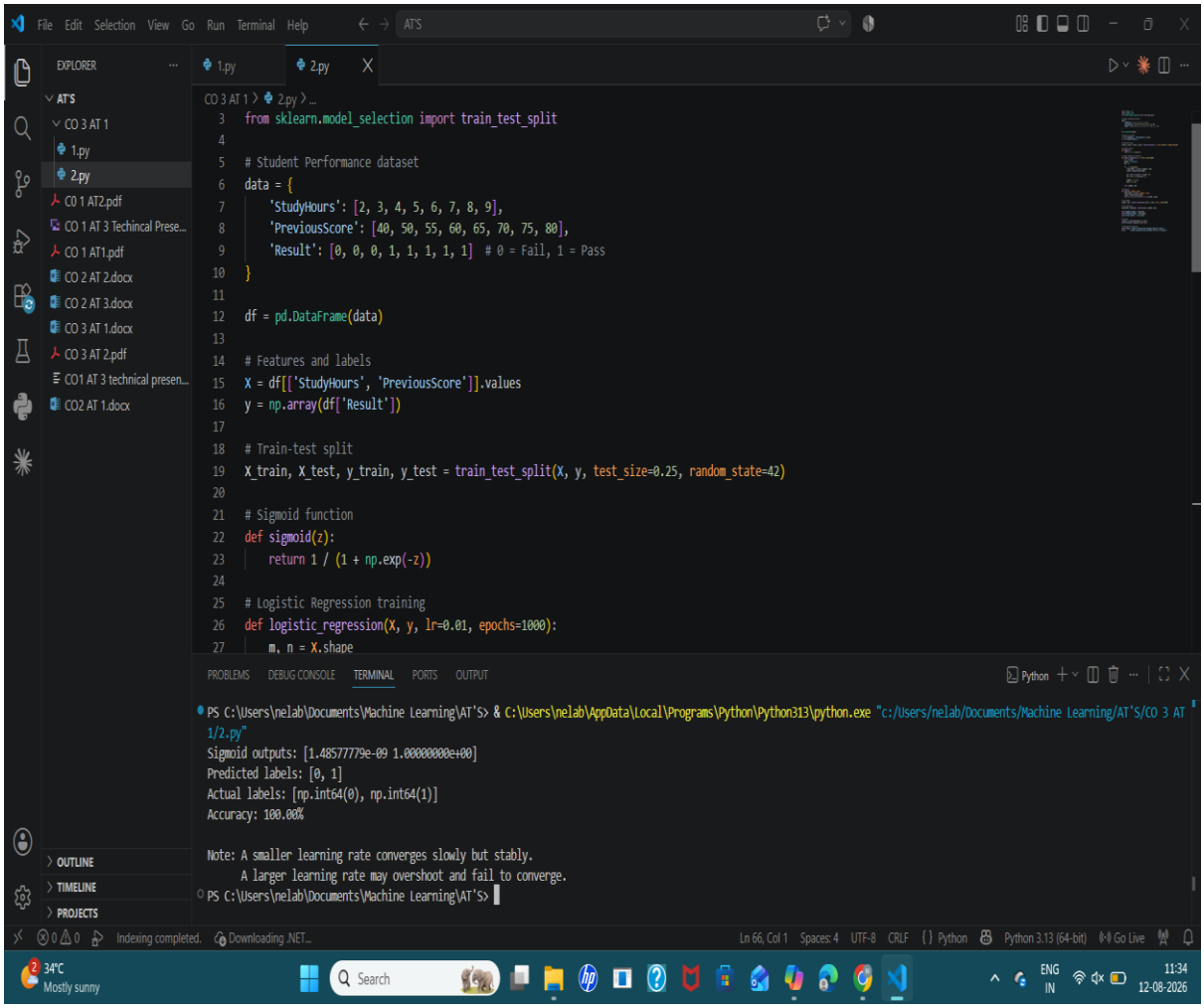
- **Small learning rate (0.001):** Slow convergence, stable training.
- **Optimal learning rate (0.1):** Smooth convergence, good accuracy.
- **Large learning rate (1.0):** Overshooting, oscillations, poor convergence.

Advantages

- Probabilistic interpretation (gives confidence scores).
- Efficient for linearly separable data.
- Easy to implement and interpret.

Limitations

- Assumes linear decision boundary.
- Not suitable for complex non-linear problems without feature engineering.


File Edit Selection View Go Run Terminal Help
AT'S
EXPLORER
AT'S
CO 3 AT 1
1.py
2.py
CO 1 AT2.pdf
CO 1 AT 3 Techincal Prese...
CO 1 AT1.pdf
CO 2 AT 2.docx
CO 2 AT 3.docx
CO 3 AT 1.docx
CO 3 AT 2.pdf
CO1 AT 3 technical presen...
CO2 AT 1.docx
OUTLINE
TIMELINE
PROJECTS
1.py
2.py
CO 3 AT 1 > 2.py > ...
3 from sklearn.model_selection import train_test_split
4
5 # Student Performance dataset
6 data = {
7 'StudyHours': [2, 3, 4, 5, 6, 7, 8, 9],
8 'PreviousScore': [40, 50, 55, 60, 65, 70, 75, 80],
9 'Result': [0, 0, 0, 1, 1, 1, 1, 1] # 0 = Fail, 1 = Pass
10 }
11
12 df = pd.DataFrame(data)
13
14 # Features and labels
15 X = df[['StudyHours', 'PreviousScore']].values
16 y = np.array(df['Result'])
17
18 # Train-test split
19 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
20
21 # Sigmoid function
22 def sigmoid(z):
23 return 1 / (1 + np.exp(-z))
24
25 # Logistic Regression training
26 def logistic_regression(X, y, lr=0.01, epochs=1000):
27 m, n = X.shape
PROBLEMS DEBUG CONSOLE TERMINAL PORTS OUTPUT
Python
PS C:\Users\nelab\Documents\Machine Learning\AT'S> & C:\Users\nelab\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/nelab/Documents/Machine Learning/AT'S/CO 3 AT 1/2.py"
Sigmoid outputs: [1.48577779e-09 1.00000000e+00]
Predicted labels: [0, 1]
Actual labels: [np.int64(0), np.int64(1)]
Accuracy: 100.00%
Note: A smaller learning rate converges slowly but stably.
A larger learning rate may overshoot and fail to converge.
PS C:\Users\nelab\Documents\Machine Learning\AT'S>
Indexing completed.
Downloading .NET...
Ln 66, Col 1 Spaces: 4 UTF-8 CRLF Python Python 3.13 (64-bit) Go Live
34°C
Mostly sunny
Search
ENG
IN
11:34
12-08-2026